

Python을 이용한 파일 처리

02 간단하게 사용할 수 있는 파일 구조

목차 02 간단하게 사용할 수 있는 파일 구조

1. 표 자료를 저장하는 CSV
2. 연관 정보를 저장하는 JSON

1. 표 자료를 저장하는 CSV

```
12345,Smith,John,"3, North Street"  
41352,Jones,Ivor,"5, South Road"  
53421,Edwards,David,"10, East Lane"
```

- 쉼표로 구분된 값(comma-separated values)으로 구성된 파일, .csv 확장자를 가짐
- 표를 나타내는 표준화된 방법 중 가장 단순한 방법
- 형식이 단순하고 작성하기 쉬우나 정렬되지 않아 가공되지 않은 형태로는 사람이 읽기 어려울 수 있음
 - 단순한 형태이므로 컴퓨터로 간단하게 읽기 쉽게 바꿀 수 있음, Excel이 지원하는 파일 형식
- 자료 자체에 쉼표가 있는 경우 별도의 구분 기호가 필요함



CSV 자료를 Python으로 표현하기

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

1. 표 자료를 저장하는
CSV

```
column_1,column_2,column_3,column_4
1,6,11,16
2,7,12,17
3,8,13,18
4,9,14,19
5,10,15,20
```

CSV 자료

```
{
    "column_1": [1, 2, 3, 4, 5],
    "column_2": [6, 7, 8, 9, 10],
    "column_3": [11, 12, 13, 14, 15],
    "column_4": [16, 17, 18, 19, 20],
}
```

Python 딕셔너리

{열 이름: 자료 리스트} 형태

CSV 자료를 대각선으로 뒤집은 것과 같음



Python으로 CSV 쓰기 비교

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

1. 표 자료를 저장하는
CSV

```
import csv

table = [[1, 10, 100, 1000], [2, 20, 200, 2000]]

with open("table.csv", "w", encoding="utf-8", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(table)

---
```

1,10,100,1000
2,20,200,2000

Python 내장 라이브러리 사용

```
import pandas as pd

df = pd.DataFrame([[1, 10, 100, 1000], [2, 20, 200, 2000]])

df.to_csv("table.csv")

---
```

,0,1,2,3
0,1,10,100,1000
1,2,20,200,2000

pandas 사용 (패키지 설치 필요)

위에 머릿글, 왼쪽에 인덱스열 삽입됨



```
import pandas as pd

df = pd.read_csv("file1.csv")
df.to_csv("file2.csv")
```

- **pd.read_csv**: pandas를 이용하여 CSV를 읽기
 - **open** 함수와 달리 **encoding**을 지정하지 않으면 기본적으로 UTF-8을 사용
- **df.to_csv**: pandas의 DataFrame을 CSV로 변환하여 저장
 - **pd.read_csv**와 마찬가지로 기본적으로 UTF-8 사용

- Python의 딕셔너리로 열(column) 구성하기
 - 딕셔너리의 키를 열 이름으로, 값을 자료 리스트로 설정
- 만들어진 딕셔너리를 `pd.DataFrame`으로 감싸 pandas 자료형으로 만든 후 `to_csv` 메서드를 이용하여 파일로 저장
- pandas로 만든 CSV는 인덱스를 나타내는 열이 자동으로 삽입됨
 - 인덱스 열을 추가하지 않으려면 `to_csv` 메서드의 `index` 인자를 `False`로 설정



pandas를 이용하여 CSV 쓰기

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

1. 표 자료를 저장하는
CSV

```
import pandas as pd

df = pd.DataFrame(
    {
        "name": ["Zane Cote", "Jolie Hewitt", "Sylvia Reese"],
        "phone": ["0500 055670", "07861 412407", "056 3915 6966"],
    }
)
df.to_csv("file.csv")
```

Python 딕셔너리로 구성한 표

	name	phone
---	-----	-----
0	Zane Cote	0500 055670
1	Jolie Hewitt	07861 412407
2	Sylvia Reese	056 3915 6966

DataFrame으로 변환된 표의 실제 내용

```
import pandas as pd

df = pd.read_excel("file.xlsx")
print(df)
```

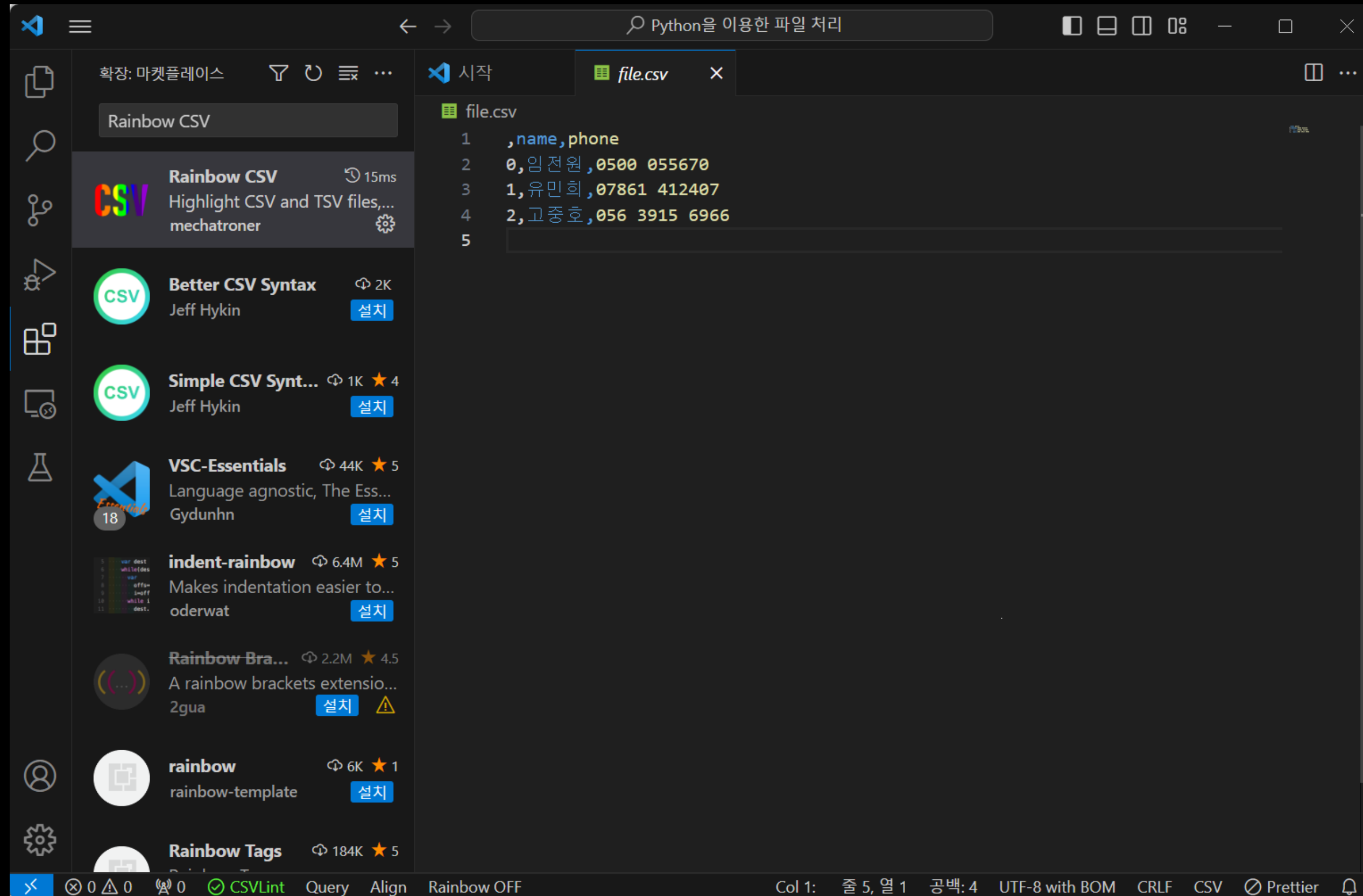
#	name	phone
# 0	임전원	0500 055670
# 1	유민희	07861 412407
# 2	고중호	056 3915 6966

pandas를 이용하면 CSV 이외에도
Excel 등과 같은 파일 형식의 읽기와 쓰기가 가능
읽기: `read_excel`
쓰기: `to_excel`

```
import pandas as pd

df = pd.DataFrame(
    {
        "name": ["임전원", "유민희", "고중호"],
        "phone": ["0500 055670", "07861 412407", "056 3915 6966"],
    }
)
df.to_csv("file.csv", encoding="utf-8-sig")
```

`to_csv`는 기본 인코딩이 UTF-8이나 만약
CSV 파일을 Excel에서 문제 없이 읽기 위해서는
"utf-8-sig"로 인코딩 지정 필요
(읽을 때는 지정 필요 없음)



Visual Studio Code의 확장

Rainbow CSV를 이용하면

단순 CSV 파일을 열별로 색상을

변경하여 구분을 편하게 만들어줌

2. 연관 정보를 저장하는 JSON



- JavaScript의 객체 문법(JavaScript Object Notation)을 따르는 문자 기반의 자료 표현
- 6개의 자료형으로 구성
 - 대표적인 자료형인 object는 키: 값 쌍으로 구성
 - Python의 dict와 거의 1:1 대응 가능
- 사람이 읽을 수 있고 컴퓨터가 처리하기도 편리
- 웹에서 정보를 전송하거나 프로그램 설정 등을 구성할 때 주로 사용함



```
[
  {
    "name": "Molecule Man",
    "age": 29,
    "secretIdentity": "Dan Jukes",
    "powers": ["Radiation resistance", "Turning tiny", "Radiation blast"]
  },
  {
    "name": "Madame Uppercut",
    "age": 39,
    "secretIdentity": "Jane Wilson",
    "powers": [
      "Million tonne punch",
      "Damage resistance",
      "Superhuman reflexes"
    ]
  }
]
```



• 자료형 대응

• dict – object

- Python의 dict와 동일한 표기 사용, 단, object의 키는 항상 문자열이어야 함

• list, tuple – array

- Python의 list와 동일한 표기 사용

• str – string

- Python의 str과 동일한 표기 사용, 단, 감싸는 따옴표는 항상 큰따옴표여야 함

• int, float – number

- Python과 달리 둘을 자료형으로써 구분하지 않음

• True – true

• False – false

• None – null



```
# Python 딕셔너리
{
    "name": "Elice",
    "language": "Python", # 후행 쉼표 허용
}

# 아래의 JSON은 후행 쉼표도, 주석도 작성 불가능
---

{
    "name": "Elice",
    "language": "Python"
}
```

• 공통점

- 딕셔너리와 JSON object 모두
{키: 값} 형태의 표기 사용

• 차이점

- JSON 자체는 자료형이 한정되지 않음
- JSON은 후행 쉼표가 허용되지 않음
- JSON에서 주석을 사용할 수 없음



- 유연한 데이터 표현
 - CSV 및 표는 2차원의 데이터만 표현 가능
 - 하나의 행에는 하나의 자료만 저장 가능
 - JSON은 유연한 데이터 표현이 가능
 - 하나의 키에 대해 단일 자료가 아닌 object, array 등으로 확장 가능
 - 중첩하여 접근(`d["key"][1]["key"]`)
- 설정 파일 구성
 - JSON에 설정을 구성하여 다음과 같은 번거로운 작업을 대체할 수 있음
 - 실행할 때마다 프로그램의 일부를 변경해야 함
 - input 함수 등으로 반복적인 내용을 입력해야 함



```
send_method = input("문서를 어떻게 전송할까요? (1: 이메일, 2: MMS): ")
if send_method == "1":
    send_email(email_list)
else:
    send_mms(phone_list)
```

프로그램을 실행할 때마다 입력해야 함

config.json:

```
{ "send_method": 1 }
```

```
with open("config.json", encoding="utf-8") as configfile:
    config = json.load(configfile)
if config["send_method"] == 1:
    send_email(email_list)
else:
    send_mms(phone_list)
```

이미 저장된 값을 읽어 실행

- Python에 내장된 라이브러리
 - `import json`을 선언하여 이용 가능
- `json.dump(obj, fp)`
 - 호환 가능 자료형(`obj`)을 JSON으로 변환하여 파일(`fp`)에 쓰기
- `json.dumps(obj)`
 - 호환 가능 자료형(`obj`)을 JSON으로 변환하여 문자열로 반환
- `json.load(fp)`
 - JSON이 저장된 파일(`fp`)을 Python 자료형으로 변환하여 반환
- `json.loads(s)`
 - JSON이 저장된 문자열(`s`)을 Python 자료형으로 변환하여 반환



json.dump 사용 예제

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

2. 연관 정보를 저장하는
JSON

```
import json

payload = {
    "method": "POST",
    "mode": "cors",
    "cache": "no-cache",
    "credentials": "same-origin",
    "headers": {"Content-Type": "application/json"},
    "redirect": "follow",
    "referrerPolicy": "no-referrer",
    "data": {"answer": 42},
}

print(json.dumps(payload))
# {"method": "POST", "mode": "cors", "cache": "no-cache", "credentials": "same-origin", "headers": {"Content-Type": "application/json"}, "redirect": "follow", "referrerPolicy": "no-referrer", "data": {"answer": 42}}
with open("payload.json", "w", encoding="utf-8") as f:
    json.dump(payload, f)
# json.dumps(payload)의 결과를 "payload.json"에 저장
```



json.load 사용 예제

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

2. 연관 정보를 저장하는
JSON

```
{
  "method": "POST",
  "mode": "cors",
  "cache": "no-cache",
  "credentials": "same-origin",
  "headers": { "Content-Type": "application/json" },
  "redirect": "follow",
  "referrerPolicy": "no-referrer",
  "data": { "answer": 42 }
}
```

payload.json

```
import json

with open("payload.json", encoding="utf-8") as f:
    print(json.load(f))
    # {'method': 'POST', 'mode': 'cors', 'cache': 'no-cache', 'credentials': 'same-origin', 'headers': {'Content-Type': 'application/json'}, 'redirect': 'follow', 'referrerPolicy': 'no-referrer', 'data': {'answer': 42}}
    f.seek(0)
    print(json.loads(f.read()))
    # 위의 결과와 같음
```

Python으로 payload.json 파일을 읽어

Python 객체로 반환

추가적으로 알아두면 좋은 매개변수(dump)

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

2. 연관 정보를 저장하는
JSON

```
print(json.dumps(payload, indent=4))
# {
#     "method": "POST",
#     "mode": "cors",
#     "cache": "no-cache",
#     "credentials": "same-origin",
#     "headers": {
#         "Content-Type": "application/json"
#     },
#     "redirect": "follow",
#     "referrerPolicy": "no-referrer",
#     "data": {
#         "answer": 42
#     }
# }
```

indent를 양의 정수로 지정하면
읽기 좋게 줄 바꿈과 들여쓰기를 추가
들여쓰기 수준: **indent**의 값

추가적으로 알아두면 좋은 매개변수(dump)

Python을 이용한 파일
처리

02 간단하게 사용할 수
있는 파일 구조

2. 연관 정보를 저장하는
JSON

```
import json

list_ = ["한국어"]
json_ensure_ascii = json.dumps(list_)
print(json_ensure_ascii)
# ["\ud55c\uad6d\uc5b4"]
json_not_ensure_ascii = json.dumps(list_, ensure_ascii=False)
print(json_not_ensure_ascii)
# ["한국어"]
print(json.loads(json_ensure_ascii))
# ['한국어']
print(json.loads(json_not_ensure_ascii))
# ['한국어']
```

JSON은 웹에서 정보 교환에 사용되는 형식이어서 Python에서 JSON 변환시 호환성을 위해 모든 문자를 ASCII 호환 문자로 변환함

`ensure_ascii`에 `False`를 전달하면 변환을 하지 않음

Python으로 읽을 때 다시 변환되지만 JSON 파일을 직접 봐야 하는 환경에서 유용